

CellSim: A Domain-Driven Software Framework for Stochastic Tumour Evolution Simulation

Introduction and Related Work (Draft)

May 1, 2026

Contents

1	Introduction	3
2	Related Work	4
2.1	Mathematical Models in Cancer Biology	4
2.2	Bioinformatics, Software Engineering, and the Reproducibility Challenge	6
2.3	Towards Integrated Frameworks: CellSim as a Proof of Concept . . .	7
3	CellSim: Scope, Goals, and Contributions	9
3.1	What CellSim Is—and What It Is Not	9
3.2	What CellSim Seeks to Demonstrate	9
3.3	What Does CellSim Represent? The Biological Model	10
3.4	What CellSim Achieves	11
3.5	Main Components	12
4	Reproducibility Through Automated Testing	13
4.1	Unit Testing	14
4.2	Functional Validation: The Scenario Suite	14
4.3	Initial Calibration of θ_{D_1} and θ_{D_2}	15
4.4	Containerisation and Portability	16

5	Conclusions	17
5.1	Summary	17
5.2	Limitations	17

1 Introduction

Cancer is not a single disease but a heterogeneous family of conditions sharing a common hallmark: the progressive and irreversible corruption of the cellular mechanisms that govern proliferation, differentiation, and death [20, 2][6, 7]. Tumour development is inherently a multi-scale, stochastic process that simultaneously spans the molecular level—where individual DNA strand breaks, gene silencing events, and protein–protein interactions occur—the cellular level—where fitness differences between clones are resolved through competition—and the tissue level—where the tumour microenvironment, immune surveillance, and spatial constraints shape the evolutionary landscape [18, 8]. This complexity is not merely descriptive: it has direct clinical consequences. Interventions that succeed *in vitro* routinely fail *in vivo*, and population-level epidemiological trends do not reliably predict individual-patient outcomes, precisely because the stochastic nature of early oncogenesis is systematically underestimated [2].

In response to this challenge, the past six decades have witnessed an accelerating effort to bring mathematical and computational tools to bear on cancer biology [17, 2]. The aspiration is compelling: if cancer follows deterministic or stochastic laws at the cellular level, then those laws should be amenable to formal analysis, and models derived from them should generate testable, quantitative predictions. The results, however, have been mixed. As Chauviere and colleagues noted in their critical appraisal of the field, “*over-optimistic estimations about [mathematical oncology’s] ability have created unrealistic expectations*” [2]. May articulated the underlying methodological trap with even greater precision: models are frequently constructed with “*an excruciating abundance of detail in some aspects, whilst other important facets of the problem are misty*” [12]. A model that is mathematically sophisticated yet biologically misspecified does not merely fail to predict—it may actively mislead experimental design and clinical decision-making.

The failure modes of incorrect models are not merely theoretical. A deterministic ordinary differential equation (ODE) system may capture average tumour growth dynamics while systematically missing the stochastic rare events—a single cell acquiring a second loss-of-function allele in a tumour suppressor gene, for instance—that are responsible for the transitions between disease phases [4, 13]. Cell–cell communication networks introduce frequency-dependent fitness effects into the competitive landscape of the tissue: the selective advantage of an immunosuppressive phenotype is not constant but depends on the current composition of the tumour microenvironment [18, 15]. These phenomena demand stochastic, agent-level modelling frameworks

rather than mean-field approximations.

Despite the recognised need, the translation from sound mathematical theory to reproducible, extensible, and clinically applicable software remains incomplete. The bioinformatics community has documented this gap extensively: technical debt, lack of software engineering discipline, and absent continuous integration practices compromise the reproducibility of computational findings and prevent their adoption in clinical workflows [11]. The problem is not a shortage of mathematical ideas—the theoretical foundations are deep and well-established—but rather a shortage of software artefacts that implement those ideas with the rigour, transparency, and modularity that translational research demands.

This paper presents CELLSIM, a domain-driven, open-source software framework for stochastic tumour evolution simulation, as a step towards closing this gap. The source code is publicly available at <https://github.com/lpalomerol/cellSim>. CELLSIM is designed as a proof of concept for the intersection of rigorous mathematical modelling, biologically grounded cellular mechanics, and production-quality software engineering. The remainder of this section motivates the three principal contributions of the framework. Section 2 reviews the relevant literature across the three axes—mathematical models, biological mechanisms, and software engineering practice—and positions CELLSIM within that landscape.

2 Related Work

2.1 Mathematical Models in Cancer Biology

Mathematical modelling in oncology has a history of over sixty years. Bibliometric analysis confirms that the field has grown substantially in both volume and interdisciplinary reach, with strong co-authorship networks spanning mathematics, physics, and clinical oncology [17]. The diversity of modelling approaches reflects the multi-scale nature of the underlying biology: deterministic models based on ordinary or partial differential equations have been used to describe macroscopic tumour growth dynamics and treatment response [2]; agent-based models (ABMs) capture spatial heterogeneity and clonal competition at the cellular level; and stochastic frameworks address the inherently probabilistic nature of molecular-level events.

Among stochastic models, two foundational contributions stand out. Moran’s 1958 model of random processes in genetics [13] established the mathematical framework for gene-frequency dynamics in a finite, fixed-size population. In Moran’s formulation,

instead of the simultaneous generational replacement assumed by the Wright–Fisher model, births and deaths occur as individual, asynchronous events: at each step, one individual is chosen uniformly at random to die and is replaced by the offspring of a randomly sampled survivor. This single modification—substituting individual stochastic replacement for synchronous generation turnover—has profound consequences for evolutionary dynamics: the rate of approach to homozygosity is exactly twice that of the Wright model, because the random choice of the individual to die introduces an additional source of variance [13]. Translated to oncology, the Moran process provides a rigorous framework for modelling clonal competition in homeostatic tissue, where cell number is regulated and each division event is coupled to a death event. The probability that a single mutant cell—say, one carrying a loss-of-function allele in *TP53*—ultimately fixes in the tissue, and the expected time to fixation or loss, are directly computable from Moran’s results.

Gillespie’s Stochastic Simulation Algorithm (SSA) [4] addressed a complementary problem: how to perform exact numerical simulation of coupled stochastic reaction systems in continuous time. Gillespie’s key insight was philosophical as much as technical. As he wrote, “*the stochastic formulation of chemical kinetics has a firmer physical basis than the deterministic formulation*”, because the deterministic rate equation is merely the expected-value projection of the underlying master equation [4]. The SSA generates statistically exact trajectories of the master equation without solving it analytically, making it possible to simulate rare molecular events—the very events responsible for somatic mutation and phenotypic switching—with mathematical exactness. This is precisely the regime where ODE-based models are least reliable: the second loss-of-function mutation in a tumour suppressor gene affecting a small number of cells in a large tissue is a rare event that ODE approximations systematically misrepresent.

When cellular fitness is not constant but depends on the current composition of the population—as occurs in the tumour microenvironment, where immunosuppressive cells alter the selective landscape for neighbouring cells [18]—the appropriate theoretical framework is evolutionary game theory. Nowak and Sigmund demonstrated that evolutionary dynamics can be frequency-dependent in a manner that is analytically tractable using replicator equations and evolutionary stable strategy analysis [15]. The implication for tumour modelling is direct: the competitive advantage of a cell with an immunoevasive phenotype is not fixed, but depends on the proportion of such cells already present in the tissue, a feedback that purely kinetic models cannot capture. Nowak’s comprehensive treatment of evolutionary dynamics [14] further established that fixation probabilities and times in finite populations under

selection—central quantities for understanding clonal expansion—are analytically accessible within these frameworks.

Despite this strong theoretical foundation, Hassan and colleagues [8] identify a persistent gap between mathematical models and clinical applicability: most models are developed and validated on *in silico* or cell-line data without external validation against independent clinical cohorts. The combination of machine learning and mechanistic mathematical modelling holds promise for closing this gap, but the technical prerequisites—software frameworks that are both mathematically rigorous and experimentally interoperable—are still largely absent.

2.2 Bioinformatics, Software Engineering, and the Reproducibility Challenge

A second body of literature relevant to this work concerns not the mathematical content of models but their software realisation. The rapid growth of biological data and the proliferation of computational tools have created a secondary crisis: much of the software produced by researchers is unreproducible, undocumented, and impossible to extend or validate independently [11]. Ma and colleagues [11] provide a systematic review of bioinformatics software development practices, identifying technical debt—the accumulated cost of shortcuts taken during development—as the primary obstacle to reproducibility. Their analysis identifies several critical deficiencies: absence of automated testing, lack of version control discipline, hardcoded parameters that prevent systematic sensitivity analysis, and monolithic architectures that resist extension or combination with other tools.

Veneziano and colleagues [20] document the practical consequences of this gap in the context of breast cancer bioinformatics: despite the availability of rich transcriptomic, genomic, and proteomic data, the field lacks software tools capable of integrating multi-omic signals into coherent mechanistic narratives. The gap is not conceptual but infrastructural.

Thomas and Barry [19] argued over two decades ago for a domain-oriented approach to software development in which software abstractions are organised around the conceptual vocabulary of the domain—in this case, cell biology and mathematical genetics—rather than around computational convenience. Their model-driven, domain-oriented programming (DOP) framework anticipates what is now called Domain-Driven Design (DDD) [19]: the principle that the structure of the software should reflect the structure of the domain it models, making the codebase intelli-

gible to biologists, mathematicians, and software engineers alike. This approach is particularly relevant for scientific computing, where the primary consumer of the software is often a domain expert who needs to reason about model behaviour without reverse-engineering implementation details.

Although the precise magnitude of the reproducibility crisis in science remains contested—Fanelli argues that the evidence does not support a systemic collapse of research reliability [3], and Pethick and colleagues demonstrate that even dedicated policy reports lack the quantitative baseline needed to assess whether a crisis truly exists [16]—the debate itself underscores a methodological imperative: regardless of scale, preventable reproducibility failures are unacceptable in research with clinical implications. Best practices such as automated testing, version control, open data policies, and transparent architectural documentation are justified not by the assumption of widespread misconduct, but by the lower bound they place on avoidable error. CELLSIM adopts these practices from the outset, treating reproducibility not as a response to crisis but as a minimum standard of scientific rigour.

The intersection of the two deficits—missing mathematical rigour in implementations, and missing software quality in mathematical tools—defines the space that the present work aims to address.

2.3 Towards Integrated Frameworks: CellSim as a Proof of Concept

The preceding review identifies a three-way gap at the intersection of cancer biology, stochastic mathematical modelling, and software engineering. Mathematical oncology has produced powerful theoretical tools—the Moran process, the Gillespie SSA, evolutionary game theory—but these tools remain largely disconnected from software artefacts that implement them in a reproducible, extensible, and biologically grounded way. Pugh and colleagues [17] confirm that interdisciplinarity is the defining characteristic of successful mathematical oncology research, yet the software infrastructure required to support multidisciplinary collaboration is systematically underdeveloped.

CELLSIM is designed as a proof of concept for an integrated approach to this problem. It is an open-source, C++20 cellular evolution simulator that adopts a hexagonal (ports-and-adapters) architecture [19], organising all domain logic—cell lifecycle, genetic state, signal integration—in a pure domain layer that depends on no external framework and is independently testable.

The biological model at the core of **CELLSIM** is grounded in established cancer genetics. Each **AgenticCell** carries structured representations of tumour suppressor genes (*TP53*, *BRCA1*) with biologically constrained state transitions: allelic states progress strictly from wild-type (+/+) through heterozygosity (+/-) to homozygous loss (-/-), consistent with the two-hit model of tumour suppressor inactivation [9]. Two continuous damage accumulators— D_1 for DNA damage and D_2 for immuno-suppressive signalling—mediate the transition between six biologically defined cell life stages (BASELINE $\rightarrow$ UNSTABLE $\rightarrow$ UNPROTECTED $\rightarrow$ PRIMER $\rightarrow$ TUMORAL).¹ This staging mirrors the multi-step model of carcinogenesis documented in the literature [2, 20].

The cell lifecycle is implemented as a strictly ordered sequence of biological decisions: intrinsic apoptosis mediated by *TP53* is evaluated before extrinsic immune-mediated apoptosis, which precedes neoplastic development and cytoplasmic remodelling leading to division. This ordering is not arbitrary but reflects the known hierarchy of cell fate decisions in oncogenesis [18].

From an engineering perspective, **CELLSIM** implements continuous integration with a dual-compiler matrix (GCC and Clang) and a comprehensive unit-test suite. All architectural decisions are documented as Architecture Decision Records (ADRs), providing a transparent audit trail that is essential for scientific reproducibility and for onboarding contributors from different disciplinary backgrounds.

Critically, the architecture is designed with extensibility as a first-class concern, providing a foundation from which mathematically rigorous stochastic frameworks—such as the Moran process, the Gillespie SSA, or tau-leaping approximations—could be integrated in future iterations [13, 4, 5]. The present version constitutes a deliberate proof of concept focused on software structure: a clean domain model, strict layer separation, and a comprehensive test suite establish the engineering substrate on which such extensions can be built incrementally and validated independently. The value of this contribution lies not in the enumeration of planned features but in the rigour of the foundation—a codebase that can grow without accumulating the technical debt that, as the literature reviewed in Section 2.2 documents, has historically undermined reproducibility in computational biology.

Together, these components instantiate the three-way integration that the preceding literature review identifies as necessary but currently absent: a biologically grounded domain model, implemented with production-quality software engineering, that can accommodate the full range of established stochastic mathematical frameworks for

¹The remaining stage is **DEAD**.

tumour evolution.

3 CellSim: Scope, Goals, and Contributions

3.1 What CellSim Is—and What It Is Not

CELLSIM is a proof of concept at the intersection of software engineering and computational biology. Its primary purpose is to explore whether the design principles of modern software craftsmanship—domain-driven design, hexagonal architecture, continuous integration, and systematic testing—can be applied productively to the construction of biological simulation tools, yielding artefacts that are simultaneously intelligible to domain experts and maintainable by software engineers.

It is explicitly *not* an exhaustive biological simulator. CELLSIM does not aim to reproduce the full complexity of the tumour microenvironment, to capture the quantitative dynamics of specific clinical datasets, or to replace the established computational oncology platforms that have been developed and validated over decades. Biological mechanisms are represented at a level of abstraction sufficient to exercise the software architecture and to ground the domain model in recognisable oncological concepts—no more. This intentional scope limitation is a design decision, not a deficiency: a proof of concept that tries to be everything risks demonstrating nothing.

3.2 What CellSim Seeks to Demonstrate

The central argument of CELLSIM is organisational as much as technical. Traditional approaches to scientific software development often follow a commissioning model: a research group with domain expertise outsources implementation to a software contractor or a junior developer, with the implicit assumption that the scientific knowledge and the engineering knowledge can be cleanly separated. This model systematically produces the deficits documented in Section 2.2: software that works for its original purpose but cannot be extended, validated, or understood by anyone other than its original author.

CELLSIM proposes an alternative: a genuinely multidisciplinary development process in which biologists, mathematicians, and software engineers share a common conceptual vocabulary, embodied in the domain model itself. When the software abstractions—`AgenticCell`, `Genome`, `CellLifeStage`, `CellDivisionSignal`—mirror the biological concepts they represent, domain experts can reason directly

about model behaviour, and software engineers can refactor or extend the system without requiring biological retraining. The goal is to demonstrate that this mode of collaboration is not only feasible but produces qualitatively better artefacts than the commissioning alternative.

3.3 What Does CellSim Represent? The Biological Model

Before describing what CELLSIM achieves, it is necessary to clarify *what* it simulates—the biological abstractions that anchor the domain model in recognisable oncological concepts.

Tissue. The simulation unit is a *tissue*: a finite, discrete population of cells that coexist and compete over time. At each simulation tick, every living cell executes its lifecycle, and the tissue manager handles the consequences—removing cells that undergo apoptosis and adding daughter cells produced by division. No spatial geometry is modelled; the tissue is a well-mixed population.

Cell and Genome. Each cell carries a *genome* composed of two tumour suppressor genes: *TP53* and *BRCA1*. Both genes are modelled with three discrete allelic states—wild-type (+/+), heterozygous loss (+/-), and homozygous loss (-/-)—consistent with Knudson’s two-hit hypothesis [9]. State transitions are strictly unidirectional: a gene can only lose function, never recover it. *BRCA1* is initialised in the heterozygous state (+/-) in all cells, reflecting the germline mutation present in hereditary breast and ovarian cancer carriers [20]. *TP53* starts wild-type (+/+) and acquires loss-of-function mutations stochastically during the simulation.

Damage Accumulators D_1 and D_2 . Two continuous scalar accumulators mediate the transition between cell life stages and capture biological processes that cannot be reduced to discrete allelic state alone.

D_1 represents the *accumulated DNA damage burden*: the integrated load of unrepaired DNA lesions (double-strand breaks, oxidative adducts) that accumulates when repair capacity is compromised. The two modelled genes contribute differently: *BRCA1* loss impairs homologous recombination (HR) repair directly, causing double-strand breaks to accumulate; *TP53* loss eliminates the G1/S checkpoint and transcriptional activation of repair genes, preventing arrest and repair initiation. Both deficiencies therefore accelerate D_1 accumulation, but through distinct mechanisms. D_1 increases at each tick by an amount that depends on the current genomic state; when D_1

crosses a calibrated threshold θ_{D_1} , the cell enters the **PRIMER** stage, reflecting the point at which genomic instability becomes self-sustaining [7].

D_2 represents the *immunosuppressive signalling load*: the cumulative production of immunosuppressive factors (e.g., TGF- β , IL-10, PD-L1 upregulation) that progressively shield the cell from immune-mediated elimination [18]. In reality, immunosuppressive reprogramming arises via specific oncogenic pathways—including cGAS–STING signalling, NF- κ B activation, and mismatch-repair deficiency-driven neoantigen dilution—rather than directly from tumour suppressor loss. In **CELLSIM**, the link between genomic dysfunction and D_2 accumulation is a deliberate simplification: it encodes the net immunosuppressive effect of progressing genomic instability without resolving the underlying signalling pathways. D_2 increases with genomic dysfunction; when it crosses a calibrated threshold θ_{D_2} , the cell transitions to **TUMORAL** and becomes fully resistant to extrinsic apoptosis. The specific values of θ_{D_1} and θ_{D_2} are not fixed a priori: they are derived through a calibration procedure against biological reference data, described in Section 4.3.

Stage definitions. The five life stages map to recognisable biological states as follows. **BASELINE**: the cell is in homeostatic equilibrium—*TP53* wild-type, *BRCA1* heterozygous, DNA damage within normal bounds. **UNSTABLE**: *TP53* has acquired its first hit (+/-); apoptotic competence is reduced but not abolished, and DNA repair coordination begins to deteriorate. **UNPROTECTED**: *TP53* has undergone homozygous loss (-/-), completing the two-hit inactivation; the intrinsic apoptotic checkpoint is abolished and genomic instability is no longer contained. **PRIMER**: $D_1 \geq \theta_{D_1}$; the accumulated DNA damage burden has crossed the threshold at which genomic instability becomes self-sustaining—the pre-malignant, neoplastic state. **TUMORAL**: $D_2 \geq \theta_{D_2}$; immunosuppressive signalling load is sufficient to evade immune-mediated elimination; the cell has acquired a fully malignant phenotype.

3.4 What CellSim Achieves

The present work delivers a functional prototype that validates the proposed approach. **CELLSIM** implements a complete cell lifecycle simulation—from baseline homeostasis through genomic instability, immune evasion, and tumoral progression—within a clean, layered architecture that has been verified by a comprehensive automated test suite. The codebase compiles under two independent compilers, all architectural decisions are documented as auditable Architecture Decision Records, and the simulation is configurable via a structured JSON interface that separates scientific

parameters from implementation details.

These properties are not incidental: they are the direct outcome of the design methodology under test. The prototype thus constitutes evidence for the central claim of this work— that disciplined software engineering, applied from the outset and in close collaboration with domain experts, is both compatible with and beneficial to computational biology research.

3.5 Main Components

CELLSIM is organised in three architectural layers that enforce a strict dependency hierarchy: a *domain* layer containing all biological logic, an *application* layer responsible for orchestration and configuration, and a *shared* layer providing cross-cutting utilities. No upward dependencies are permitted: the domain layer is fully independent of any framework or external tool, making it the stable, testable core of the system.

The principal domain components are:

AgenticCell The central abstraction of the simulation (see Figure 2). Each cell executes a strictly ordered lifecycle every simulation tick: baseline viability check, intrinsic apoptosis mediated by *TP53*, extrinsic immune-mediated apoptosis, neoplastic development, and cytoplasmic remodelling leading to division. This ordering reflects the known hierarchy of cell fate decisions in oncogenesis.

CellLifeStage The software encoding of the five-stage biological progression described in Section 3.3 (`BASELINE` $\rightarrow$ `UNSTABLE` $\rightarrow$ `UNPROTECTED` $\rightarrow$ `PRIMER` $\rightarrow$ `TUMORAL`), computed on-the-fly from the cell’s current genomic state and D_1/D_2 values. A separate `DEAD` state is the terminal outcome of apoptosis. Figure 1 illustrates the full state transition model, including the two apoptotic exit paths.

Genome and Gene Software implementation of the genetic model introduced in Section 3.3. Allelic state transitions are enforced as a strict unidirectional progression; no recovery path exists. Each gene’s state is queried by **AgenticCell** to determine apoptotic competence (*TP53*) and DNA repair capacity (*BRCA1*).

Tissue Container and lifecycle manager for the cell population. **Tissue** executes each cell’s lifecycle per tick, handling cell death and division as first-class domain events (`CellDeathException`, `CellDivisionSignal`) rather than ad-hoc control flow.

Signals and ports Inter-component communication uses a signal/exception protocol that keeps the domain layer decoupled from any specific orchestration strategy. Interfaces allow the injection of alternative implementations—including test doubles—without modifying domain logic.

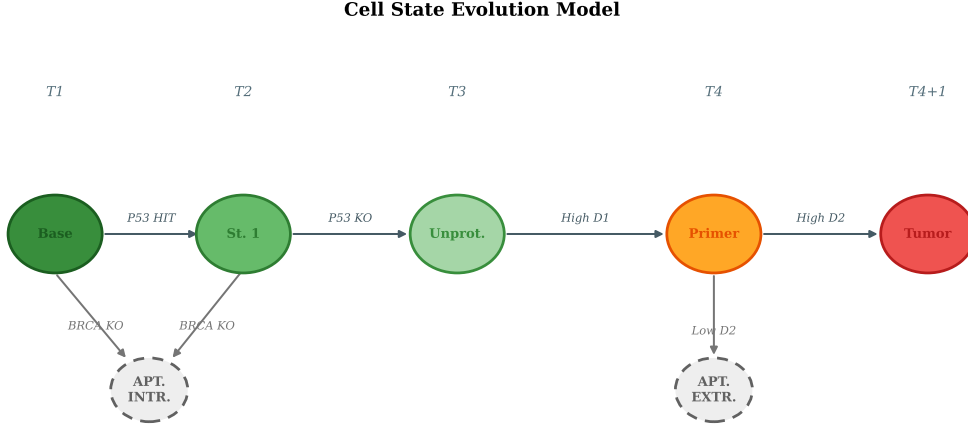


Figure 1: **Cell state transition model.** Starting from a wild-type baseline ($BRCA1^{+/-}$, $TP53^{+/+}$), successive mutation events drive the cell through five oncological stages. D_1 (DNA damage) and D_2 (immunosuppression) are continuous accumulators; threshold crossings trigger stage transitions. Two apoptotic exits are modelled: intrinsic (TP53-mediated, *Apt.Intr.*) and extrinsic / immune-mediated (*Apt.Extr.*).

The application layer exposes a JSON-configurable entry point (`cellSim_cli`) that separates scientific parameters (cell count, random seed, simulation length, mutation probabilities) from implementation details, enabling systematic sensitivity analyses without recompilation. A suite of seventeen predefined scenarios (`run_all_scenarios`) provides a validated baseline for regression testing and reproducibility audits.

4 Reproducibility Through Automated Testing

A simulation framework is only as reliable as the evidence that its behaviour is correct and stable across versions. CELLSIM treats reproducibility not as an afterthought but as a structural property of the codebase, enforced at two complementary levels: unit testing of individual domain components, and functional validation through a suite of predefined biological scenarios.

AgenticCell — 5-Phase Life Cycle

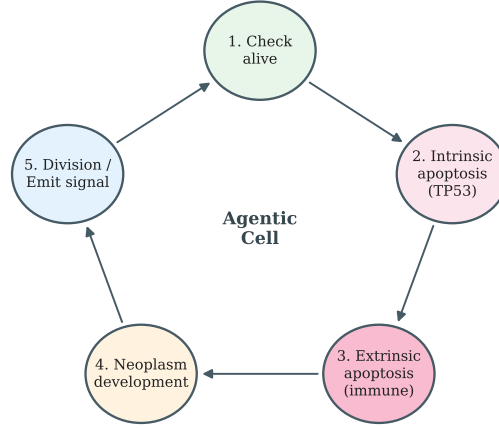


Figure 2: **AgenticCell — 5-phase ordered lifecycle.** Each simulation tick executes the five phases in strict order: viability check, intrinsic apoptosis (*TP53*), extrinsic immune-mediated apoptosis, neoplastic development, and division.

4.1 Unit Testing

The domain layer is covered by 82 unit tests across 11 test files, each targeting a specific domain concept: gene mutation and genome construction, cell lifecycle ordering and apoptosis triggering, life stage transitions, tissue population dynamics, and stochastic noise behaviour. All tests run automatically on every commit via a continuous integration pipeline under both GCC and Clang, ensuring compiler-independent correctness and immediate regression detection.

4.2 Functional Validation: The Scenario Suite

Beyond unit-level correctness, CELLSIM includes a suite of 17 predefined biological scenarios (`run_all_scenarios`) that exercise the full simulation stack end-to-end. The scenarios are organised in four groups:

Scenarios 01–09: Baseline and parametric controls Nine scenarios covering the absence of mutations, high division rates, isolated BRCA1 or TP53 mutation loads, and realistic parameter combinations. These controls establish expected behaviour boundaries—e.g., that a tissue with no mutations and no division remains stable, while high TP53 mutation rates combined with genomic instability produce neoplastic progression within biologically plausible

timeframes.

Scenarios 10–12: Big Bang tumoral dynamics Three scenarios activate the Big Bang mode, in which cells that have reached homozygous *TP53* loss acquire an accelerated division rate. These scenarios validate the emergence of clonal dominance and tumoral expansion under competitive conditions.

Scenarios 13–15: Calibrated BRCA1 carrier profile with lognormal noise

Three scenarios implement a calibrated parameter set ($\delta_{\text{low}} = 0.0317$, $\theta_{D_1} = 4.0$, $\theta_{D_2} = 10.0$) derived to reproduce a BRCA1 carrier penetrance profile, with log-normal noise at coefficient of variation 0.3 and 0.5, and at two population sizes ($N=1000$, $N=2000$). Each scenario runs 10 independent replicates, providing distributional estimates of onset age and penetrance.

Scenarios 16–17: Calibrated profile without noise Two baseline comparisons using the same calibrated parameters but with constant (deterministic) damage increments, isolating the contribution of stochastic noise to outcome variability.

Together, the unit tests and scenario suite provide two interlocking guarantees: that each component behaves correctly in isolation, and that their composition produces biologically coherent emergent dynamics. This dual-level validation strategy is the operational realisation of the reproducibility principles discussed in Section 2.2.

4.3 Initial Calibration of θ_{D_1} and θ_{D_2}

The thresholds θ_{D_1} and θ_{D_2} that govern the **PRIMER** and **TUMORAL** transitions are not derived analytically from first principles. Instead, they are calibrated against a biological reference target: the observed penetrance profile of *BRCA1* germline mutation carriers, in whom lifetime risk of breast cancer approaches 70 % with a characteristic age-of-onset distribution peaking in the fourth and fifth decades of life [1].

Temporal mapping. Each simulation tick is treated, as a first approximation, as equivalent to one year of biological time. This mapping is not derived from first principles and should be regarded as an initial proposal: it provides an order-of-magnitude anchor for comparing simulated onset distributions against clinical age-of-onset data, but formal justification—linking tick duration to a cell-division rate and a tissue renewal timescale—is deferred to future calibration work.

Calibration approach. The procedure is exploratory rather than formally statistical. A range of parameter combinations— θ_{D_1} , θ_{D_2} , and the baseline damage increment δ_{low} —are swept across biologically plausible intervals. For each combination, the scenario suite (scenarios 13–17, Section 4.2) is run with multiple independent replicates, and the resulting distribution of tumoral-onset simulation ticks is compared qualitatively against the reference penetrance curve. The parameter set that produces onset distributions most consistent with the expected 70 % lifetime penetrance and the characteristic age-of-onset profile is retained as the working calibration.

Current working values. The present implementation uses $\delta_{\text{low}} = 0.0317$, $\theta_{D_1} = 4.0$, and $\theta_{D_2} = 10.0$ as an initial calibration proposal. These values reproduce the qualitative shape of the BRCA1 carrier penetrance profile under lognormal noise at coefficient of variation $CV = 0.3\text{--}0.5$ and population sizes $N = 1000\text{--}2000$.

Limitations and future work. This calibration is a first approximation. No formal parameter-fitting algorithm (e.g., Approximate Bayesian Computation, ABC) has been applied, and no confidence intervals on the threshold estimates are available. Formal statistical calibration against independent clinical cohort data—and extension to calibrate the model against tumour types beyond hereditary breast cancer—is planned as a priority follow-on activity if this work is funded.

4.4 Containerisation and Portability

A further dimension of reproducibility is environmental: a simulation that produces correct results on the developer’s machine but cannot be executed on a collaborator’s system or an HPC cluster is not reproducible in any practically useful sense. Container technologies address this problem by encapsulating the entire runtime environment—operating system libraries, compiler runtime, binary executables, and configuration files—into a single portable image.

Two container ecosystems are relevant in this context. **Docker** is the dominant standard in general software engineering and cloud deployments, providing a layered image format and a broad ecosystem of base images and tooling. **Apptainer** (formerly Singularity [10]) is the standard in high-performance computing (HPC) environments, where Docker’s daemon-based architecture is incompatible with multi-user cluster security policies; Apptainer images run without root privileges and integrate natively with job schedulers such as SLURM.

CELLSIM provides an Apptainer definition file (`cellSim.def`) that packages all simu-

lation executables (`cellSim_cli`, `run_all_scenarios`, `interactive`, `single_cell_evolution`) together with their runtime dependencies and configuration templates into a self-contained image deployable on any Apptainer-compatible system. This makes it possible to reproduce any simulation result—including the full 17-scenario validation suite—on a remote HPC cluster without recompilation or environment configuration.

5 Conclusions

5.1 Summary

This work presents CELLSIM, a proof-of-concept framework demonstrating that the application of modern software engineering principles to computational biology is not only feasible but produces qualitatively superior artefacts compared to the ad-hoc development practices that dominate the field. By adopting domain-driven design, hexagonal architecture, continuous integration, and a comprehensive test suite, CELLSIM delivers a simulation of tumour evolution that is transparent, extensible, and reproducible—properties that are, as the literature reviewed in Section 2 documents, systematically absent from most existing computational oncology tools.

The framework validates its central organisational claim: that biologists, mathematicians, and software engineers can collaborate productively around a shared domain model, without each discipline having to sacrifice its standards to accommodate the others. The 17-scenario validation suite and the Apptainer container image operationalise reproducibility at both the algorithmic and environmental levels, making every simulation result independently verifiable on any compatible system.

5.2 Limitations

The current version of CELLSIM has several deliberate limitations that define the boundaries of its claims.

Simplified biological model The simulation models two tumour suppressor genes (*TP53* and *BRCA1*) and two damage accumulators. This is sufficient to exercise the architecture and to produce biologically recognisable dynamics, but it does not capture the full genetic landscape of any specific cancer type. No oncogenes, signalling pathways, or spatial tissue geometry are modelled.

Absence of clinical validation The calibrated BRCA1 carrier scenarios (13–17) are designed to produce penetrance profiles qualitatively consistent with the literature, but no formal fitting against clinical cohort data has been performed.

Quantitative predictions should not be extrapolated beyond the proof-of-concept scope of this work.

Single cell type All cells in the simulation share the same biological rules. Tumour microenvironment heterogeneity—including stromal cells, immune effectors, and endothelial cells—is not represented.

Stochastic framework The current noise model uses lognormal perturbations of damage increments. Mathematically rigorous stochastic frameworks such as the Moran process or the Gillespie SSA, while architecturally compatible with the codebase, have not yet been integrated.

These limitations are not unexpected for a proof of concept; they define the agenda for future development rather than deficiencies in the present contribution. The primary claim of this work—that disciplined software engineering produces better scientific software—is independent of the biological scope of the current model and is fully supported by the artefacts delivered.

References

- [1] Antonis Antoniou et al. Average risks of breast and ovarian cancer associated with BRCA1 or BRCA2 mutations detected in case series unselected for family history. *American Journal of Human Genetics*, 72(5):1117–1130, 2003.
- [2] Arnaud H. Chauviere, Haralampos Hatzikirou, John S. Lowengrub, Hermann B. Frieboes, Alastair M. Thompson, and Vittorio Cristini. Mathematical oncology: how are the mathematical and physical sciences contributing to the war on breast cancer? *Current Breast Cancer Reports*, 2(3):121–129, 2010.
- [3] Daniele Fanelli. Is science really facing a reproducibility crisis, and do we need it to? *Proceedings of the National Academy of Sciences*, 115(11):2628–2631, 2018.
- [4] Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.
- [5] Daniel T. Gillespie. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of Chemical Physics*, 115(4):1716–1733, 2001.
- [6] Douglas Hanahan and Robert A. Weinberg. The hallmarks of cancer. *Cell*, 100(1):57–70, 2000.
- [7] Douglas Hanahan and Robert A. Weinberg. Hallmarks of cancer: The next generation. *Cell*, 144(5):646–674, 2011.
- [8] Jasmin Hassan, Safiya Mohammed Saeed, Lipika Deka, Md Jasim Uddin, and Diganta B. Das. Applications of machine learning (ML) and mathematical modeling (MM) in healthcare with special focus on cancer prognosis and anticancer therapy: current status and challenges. *Pharmaceutics*, 16(2):260, 2024.
- [9] Alfred G. Knudson. Mutation and cancer: Statistical study of retinoblastoma. *Proceedings of the National Academy of Sciences*, 68(4):820–823, 1971.
- [10] Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5):e0177459, 2017.
- [11] Xu-Kai Ma, Yan Yu, Tao Huang, Dake Zhang, Caihuan Tian, Wenli Tang, Ming Luo, Pufeng Du, Guangchuang Yu, and Li Yang. Bioinformatics software development: principles and future directions. *The Innovation Life*, 2(3):100083, 2024.
- [12] Robert M. May. Uses and abuses of mathematics in biology. *Science*, 303(5659):790–793, 2004.

- [13] P. A. P. Moran. Random processes in genetics. *Mathematical Proceedings of the Cambridge Philosophical Society*, 54(1):60–71, 1958.
- [14] Martin A. Nowak. *Evolutionary Dynamics: Exploring the Equations of Life*. Harvard University Press, Cambridge, MA, 2006.
- [15] Martin A. Nowak and Karl Sigmund. Evolutionary dynamics of biological games. *Science*, 303(5659):793–799, 2004.
- [16] Stephen Pethick, Mark N. Wass, and Martin Michaelis. Is there a reproducibility crisis? On the need for evidence-based approaches. *International Studies in the Philosophy of Science*, 38(4):287–303, 2025.
- [17] Kira Pugh, Linnéa Gyllingberg, Stanislav Stratiev, and Sara Hamis. A bibliometric study on mathematical oncology: interdisciplinarity, internationality, collaboration and trending topics. *Bulletin of Mathematical Biology*, 87:174, 2025.
- [18] Jimeng Su, Ying Song, Zhipeng Zhu, Xinyue Huang, Jibiao Fan, Jie Qiao, and Fengbiao Mao. Cell–cell communication: new insights and clinical implications. *Signal Transduction and Targeted Therapy*, 9:196, 2024.
- [19] Dave Thomas and Brian M. Barry. Model driven development: the case for domain oriented programming. In *Companion to the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2003)*, pages 2–7, Anaheim, California, USA, 2003. ACM Press.
- [20] Matteo Veneziano, Isabella Savini, Elisa Cortellesi, Valeria Gasperi, Alessandra Gambacurta, and Maria Valeria Catani. Bioinformatics strategies in breast cancer research. *Biomolecules*, 15(10):1409, 2025.